

《软件安全》实验报告

姓名：谢小珂 学号：2310422 班级：1069

实验名称：

Shellcode 编写及编码

实验要求：

复现第五章实验三，并将产生的编码后的 shellcode 在示例 5-4 中进行验证，阐述 shellcode 编码的原理、shellcode 提取的思想。

实验过程：

1. 用 C 语言编写要执行的 shellcode，如图：

```
#include<stdio.h>
#include<windows.h>
void main(){
    MessageBox(NULL,NULL,NULL,0);
    return;
}
```

• 在 MessageBox(NULL,NULL,NULL,0);这行代码处设置断点，当程序运行到此处暂停后，切换到反汇编模式，查看该函数调用时生成的反汇编代码。此函数调用时总共传入了四个参数，具体情况如下：

```
5:      MessageBox(NULL,NULL,NULL,0);
00401028  mov     esi,esp
0040102A  push    0
0040102C  push    0
0040102E  push    0
00401030  push    0
00401032  call    dword ptr [__imp__MessageBoxA@16 (0042428c)]
00401038  cmp     esi,esp
0040103A  call    __chkesp (00401070)
6:      return;
7:      }
0040103F  pop     edi
00401040  pop     esi
00401041  pop     ebx
00401042  add     esp,40h
00401045  cmp     ebp,esp
00401047  call    __chkesp (00401070)
0040104C  mov     esp,ebp
0040104E  pop     ebp
0040104F  ret
```

• 在编译器中利用 _asm 内联汇编编写调用 MessageBox 函数的指令。按其调用约定将参数压栈，再调用该函数实现功能。程序正确运行后，通过对比可知，代码采用 _asm 内联汇编模拟 MessageBox 函数调用，功能与 MessageBox 函数相同。

```

#include<stdio.h>
#include<windows.h>
void main(){
    _asm{
        xor ebx,ebx
        push ebx
        push ebx
        push ebx
        push ebx
        mov eax,77d507eah
        call eax
    }
    return;
}

```

- 在 xor ebx,ebx 这一汇编指令所在行设置断点，随后切换至反汇编视图进行调试分析。

00401028 xor ebx,ebx
 9: push ebx//push 0
 0040102A push ebx
 10: push ebx
 0040102B push ebx
 11: push ebx
 0040102C push ebx
 12: push ebx
 0040102D push ebx
 13: mov eax, 77d507eah// 77d507eah 这个是MessageBox函数在系
 0040102E mov eax,77D507EAh
 14: call eax
 00401033 call eax
 15: }
 16: }
 17: }
 18: return;
 19: }
 00401035 pop edi
 00401036 pop esi

Address:	0x00401028
00401028	33 DB 53 53 53 53 B8 EA 07 D5 77 FF D0 5F 5E 5B 33
00401038	83 C4 40 3B EC E8 2E 00 00 00 8B E5 5D C3 CC CC
00401048	CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC
00401058	CC CC CC CC CC CC CC CC CC CC CC CC CC CC CC

- 可通过定位汇编语句所在的内存地址，提取该地址处对应的机器码，然后将此机器码替换到 shellcode 相应位置，进而开展调试工作。

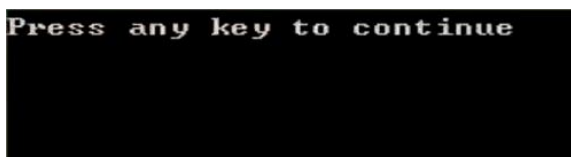
33 DB 53 53 53 53 B8 EA 07 D5 77 FF D0

```

#include<stdio.h>
#include<windows.h>
char ourshellcode[]="33\xD8\x53\x53\x53\x53\xB8\xEA\x07\xD5\xFF\xD0";
void main(){
    loadLibrary("user32.dll");
    int *ret;
    ret=(int*)&ret+2;
    (*ret)=(int)ourshellcode;
    return;
}

```

- 经测试验证，shellcode 具备调用 MessageBox 函数的能力。



2. shellcode 编写

- 下面编写 shellcode 的代码，此 shellcode 能够创建一个显示 “hello world” 的对话框，具体代码如下：

```
#include<stdio.h>
#include<windows.h>
void main(){
    loadLibrary("user32.dll");//加载user32.dll
    _asm{
        xor ebx,ebx
        push ebx//push 0
        push 20646C72h
        push 6F77206Fh
        push 6C6C6568h
        mov eax,esp
        push ebx//push 0
        push eax
        push eax
        push ebx
        mov eax,77d507eah//77d507eah是MessageBox函数在系统中的地址
        call eax
    }
    return;
}
```

提取 shellcode 代码：

\x33\xDB\x53\x68\x72\x6C\x64\x20\x68\x6F\x20\x77\x6F\x68\x68\x65\x6C\x6C\x8B\xC4\x53\x50\x50\x53\xB8\xEA\x07\xD5\x77\xFF\xD0

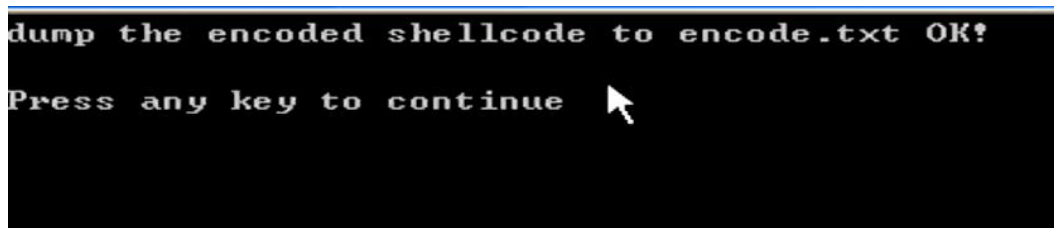
3. shellcode 编码

采用异或编码方式对 shellcode 进行处理。以下是输入所需 shellcode 后经异或编码的代码：

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
void encoder(char* input, unsigned char key)
{
    int i = 0, len = 0;
    FILE * fp;
    len = strlen(input);
    unsigned char * output = (unsigned char *)malloc(len + 1);
    for (i = 0; i < len; i++)
        output[i] = input[i] ^ key;
    fp = fopen("encode.txt", "w+");
    fprintf(fp, "");
    for (i = 0; i < len; i++)
    {
        fprintf(fp, "\\x%0.2x", output[i]);
        if ((i + 1) % 16 == 0)
            fprintf(fp, "\\n");
    }
    fprintf(fp, "");
    fclose(fp);
    printf("dump the encoded shellcode to encode.txt OK!\n");
    free(output);
}

int main(){
    char shellcode[]="\x33\xDB\x53\x68\x72\x6C\x64\x20\x68\x6F\x20\x77\x6F\x68\x68\x65\x6C\x6C\x8B\xC4\x53\x50\x50\x53\xB8\xEA\x07\xD5\x77\xFF\xD0";
    encoder(shellcode,0x44);
    getchar();
    return 0;
}
```

- 执行该代码，如图：



- 将异或操作处理后的 shellcode 编码输出至 encode.txt 文件，从而得到编码后的 shellcode。

```

'\x77\x9f\x17\x2c\x36\x28\x20\x64\x2c\x2b\x64\x33\x2b\x2c\x2c\x21'
'\x28\x28\xcf\x80\x17\x14\x14\x17\xfc\xae\x43\x91\x33\xbb\x94\xd4'
....

```

4. shellcode 解码

- 解码代码如图所示：

```

void main()
{
    __asm
    {
        add eax, 0x14
        xor ecx, ecx
    decode_loop:
        mov bl, [eax + ecx]
        xor bl, 0x44
        mov [eax + ecx], bl
        inc ecx
        cmp bl, 0x90
        jne decode_loop
    }
}

```

- 让解码器与编码后的 shellcode 一同运行，把 eax 寄存器设定为 shellcode 的起始地址。在后续操作里，对代码逐位与 0x44 进行异或运算，并用运算结果覆盖原代码。最终，在处理后的代码末尾添加 0x90 作为结束标志。

```

#include <iostream>
using namespace std;
int main(int argc, char const *argv[])
{
    unsigned int temp;
    __asm{
        call lable;
    lable:
        pop eax;
        mov temp,eax;
    }
    cout <<temp <<endl;
    return 0;
}

```


- 以汇编语句地址 00401028 为起点，获取该地址处汇编语句对应的机器码。自该起始地址起，持续提取机器码，直至结束地址 00401042 前一个地址 00401041。将提取到的机器码与 encode.txt 文件中的 shellcode 编码进行合并，最终得到完整 shellcode 编码。

```
E8\x00\x00\x00\x00\x58\x83\xC0\x15\x33\xC9\x8A\x1C\x08\x80\xF3\x44\x88\x1C\x08\x41\x80\xFB\x90\x75\xF1\x77\x9F\x17\x2C\x36\x28\x20\x64\x2C\x2B\x64\x33\x2B\x2C\x2C\x21\x28\x28\xCF\x80\x17\x14\x14\x17\xFC\xAE\x43\x91\x33\xBB\x94\xD4
```

- 在示例 5 - 4 里，用具体数值替换 ourshellcode。

```
#include<stdio.h>
#include<windows.h>
char ourshellcode[]="E8\x00\x00\x00\x00\x58\x83\xC0\x15\x33\xC9\x8A\x1C\x08\x80\xF3\x44\x88\x1C\x08\x41\x80\xFB\x90\x75\xF1\x77\x9F\x17\x2C\x36\x28\x20\x64\x2C\x2B\x64\x33\x2B\x2C\x2C\x21\x28\x28\xCF\x80\x17\x14\x14\x17\xFC\xAE\x43\x91\x33\xBB\x94\xD4";
void main(){
    LoadLibrary("user32.dll");
    int *ret;
    ret=(int*)&ret+2;
    (*ret)=(int)ourshellcode;
    return;
}
```

- 编译运行结果如图：



心得体会：

编写 Shellcode 时，对 MessageBox 函数的调用实践，让我理解了函数调用约定、参数传递机制，学会从汇编视角剖析程序底层运行逻辑。异或编码的学习，使我认识到它在隐藏和保护 Shellcode 方面的关键作用，对数据加密与处理有了更深刻的认知。在实验过程中，解决合并机器码与 Shellcode 编码等问题时，我的调试技能得到极大提升，能更熟练运用调试工具排查错误。

Shellcode 在安全测试与恶意攻击中都有潜在应用，也让我看到软件安全领域的复杂性。